

C++ Support in GCC's Static Analyzer

Progress and Future Plans

Egas Ribeiro · GNU Tools Cauldron 2026 · Prague
GSoC '26, mentored by David Malcolm

This presentation complements a write-up, which you can find at:
riogu.github.io/posts/gsoc-2026

Merged work

- Exception subclass matching: [PR119697](#)
- Virtual function devirtualization: [PR97114](#)
- `dynamic_cast` modeling: [PR110578](#)

All three are about polymorphism, and the analyzer already had most of the needed infrastructure to handle them.

How the analyzer works

Symbolic execution over gimple-SSA

```
#include <stdlib.h>

void test (int flag)
{
    void *p = malloc (16);
    if (flag)
        free (p);
    free (p);
}
```

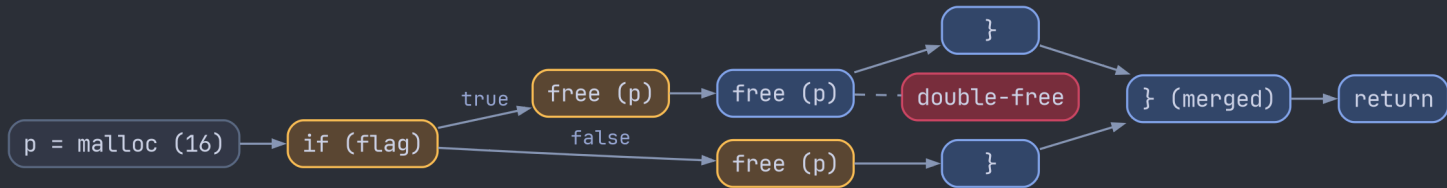
- **supergraph:** a node per gimple stmt, edges carry the operations
- **exploded graph:** nodes are `<point, state>`
- **state:** a `region_model`
 - store: regions $\Rightarrow$ svalues
 - constraint manager
- warns when a path reaches a bad state

supergraph

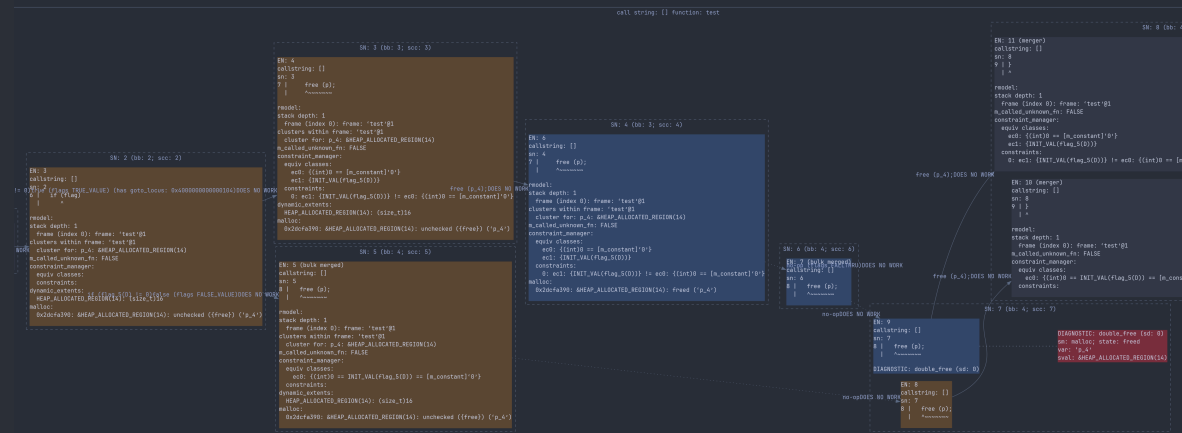
exploded graph

■ p unchecked

■ p freed



The exploded graph



GIMPLE from `-fdump-ipa-analyzer:`

```
void test (int flag)
{
  void * p;

  <bb 2> :
    p_4 = malloc (16);

    if (flag_5(D) != 0)
      goto <bb 3>; [INV]
    else
      goto <bb 4>; [INV]
```

Above graph from: `-fdump-analyzer-exploded-graph`

- one node per `<point, state>`
- p allocated, unchecked
- p freed
- the double-free diagnostic

```
void test (int flag)
{
  void *p = malloc (16);
  if (flag)
    free (p);
  free (p);
}
```

```
<bb 3> :
  free (p_4);

<bb 4> :
  free (p_4);
  return;
}
```

The store

Output of calling `__analyzer_dump ()` right after the `malloc`:

```
rmodel:
stack depth: 1
  frame (index 0): frame: 'test'@1
clusters within frame: 'test'@1
  cluster for: p_4: &HEAP_ALLOCATED_REGION(14)
m_called_unknown_fn: FALSE
constraint_manager:
  equiv classes:
  constraints:
dynamic_extents:
  HEAP_ALLOCATED_REGION(14): (size_t)16
malloc:
  0x1a01e6c0: &HEAP_ALLOCATED_REGION(14): unchecked ({free}) ('p_4')
```

The resulting diagnostic

```
double-free.c: In function 'test':
double-free.c:8:3: warning: double-'free' of 'p' [CWE-415] [-Wanalyzer-double-free]
   8 |     free (p);
     |     ^~~~~~
'test': events 1-5
   5 |     void *p = malloc (16);
     |                ^~~~~~
     |                |
     |                (1) allocated here
   6 |     if (flag)
     |     ~
     |     |
     |     (2) following 'true' branch (when 'flag != 0')... ->
   7 |         free (p);
     |         ~~~~~~
     |         |
     |         ->(3) ...to here
     |         (4) first 'free' here
   8 |     free (p);
     |     ~~~~~~
     |     |
     |     (5) ⚠ second 'free' here; first 'free' was at (4)
```

Implemented work

What was missing

Constructs that depend on an object's **runtime** type weren't modeled, just handled conservatively:

- `catch (Base &)` for a thrown `Derived` - no match
- `dynamic_cast` - unknown function, result conjured
- virtual calls - the callee was only found by evaluating the loaded function pointer, and a thunk was an unknown call that might throw

Analysis would continue past them, but with imprecise state: it would end *useful* analysis of that path.

The dynamic type is already in the model

```
struct A { int a; virtual void v () {} };  
struct B { int b; virtual void w () {} };  
struct C : A, B { int c; };
```

The constructor writes both vptrs, and the store records them like any other write:

- vtable decl - **most-derived type**
- offset - **which subobject's vptr** was read

```
__analyzer_dump () after C c = {};
```

cluster for: C c

key: {bytes 0-7}

value: 'int (*) () *' {(&constexpr int (* C::_ZTV1C [6])(...)+(sizetype)16)}

key: {bytes 16-23}

value: 'int (*) () *' {(&constexpr int (* C::_ZTV1C [6])(...)+(sizetype)40)}

The C object:		The vtable (_ZTV1C):	
0	vptr	+-----+ +0	0 (top_offset)
	+-----+	+-----+ +8	ptr to typeinfo for C
8	a	+-----+ +16	A::v()
	+-----+	+-----+ +24	-16 (top_offset)
12	(padding)	+-----+ +32	ptr to typeinfo for C
	+-----+	+-----+ +40	B::w()
16	vptr	+-----+ +48	
	+-----+		
24	b		
	+-----+		
28	c		
	+-----+		

1. Exception matching

Catching a base class

```
#include "analyzer-decls.h"

class exception {};
class io_error : public exception {};

int test ()
{
    try {
        throw io_error ();
    } catch (exception &exc) {
        __analyzer_dump_path ();
        return -1;
    }

    return 0;
}
```

Before: handler never matches, no path reaches it

After:

```
exception.cc:11:26: note: path
    11 |     __analyzer_dump_path ();
        |     ~~~~~^~
exception.cc:9:21: note: (1) throwing exception of type 'io_error' here...
     9 |     throw io_error ();
        |           ^
exception.cc:10:23: note: (2) ...catching exception of type 'io_error' here
    10 |     } catch (exception &exc) {
        |           ^~~
exception.cc:11:26: note: (3) here
    11 |     __analyzer_dump_path ();
        |     ~~~~~^~
```

Implementation

Before (analyzer/ops.cc):

```
static bool
exception_matches_type_p (tree exception_type,
                          tree catch_type)
{
    if (catch_type == exception_type)
        return true;
    /* TODO (PR analyzer/119697): we should also
       handle subclasses etc; [...]

    It looks like we should be calling (or
    emulating) can_convert_eh from the C++ FE,
    but that's specific to the C++ FE. */

    return false;
}
```

After: ask the frontend through a langhook

```
static bool
exception_matches_type_p (tree handler_type,
                          tree exception_type)
{
    return lang_hooks.exception_matches_type_p
        (handler_type, exception_type);
}
```

```
/* langhooks-def.h */
#define LANG_HOOKS_EXCEPTION_MATCHES_TYPE_P \
    hook_bool_tree_tree_false
/* cp/cp-lang.cc */
#define LANG_HOOKS_EXCEPTION_MATCHES_TYPE_P \
    handler_match_for_exception_type
```

The analyzer stays language-agnostic and gets all of [except.handle]: ambiguous and private bases, cv-quals, pointer conversions, nullptr.

Limitation: one global lang_hooks, so mixed C/C++ LTO only gets one frontend's rules.

2. Devirtualization

A false positive through a thunk

```
struct A { virtual void f () {} };
struct B { virtual void g () { throw 1; } };
struct C : public A, public B {
    void f () {}
    void g () {}
};
__attribute__((noipa)) C *make_c () { return new C (); }
```

```
void test ()
{
    C *c = make_c ();
    B *b = c;
    b->g ();
    delete c;
}
```

Before (warning and the last two events):

```
devirt.cc:16:8: warning: leak of 'c' [CWE-401] [-Wanalyzer-malloc-leak]
   16 |     b->g ();
      |     ~~~~~^~
devirt.cc:16:8: note: (10) if 'virtual void C::_ZThn8_N1C1gEv()' throws an exception...
   16 |     b->g ();
      |     ~~~~~^~
devirt.cc:16:8: note: (11) 'c' leaks here; was allocated at (4)
```

After: no warning (`b->g ();` is known to not throw)

Implementation

- `OBJ_TYPE_REF` call - read the value bound to the object's vptr field
- if it has the form `&vtable_decl + CST: gimple_get_virt_method_for_vtable (token, vtable, offset) - ultimate_alias_target`
- conjured vptr or non-constant offset - dynamic type unknown, call stays unresolved
- `can_throw_p` follows thunks through `function_symbol ()` and checks `TREE_NOTHROW`, so `noexcept` is read off the real override

None of this is really C++-specific: it only uses `OBJ_TYPE_REF` and the GIMPLE vtable representation.

3. dynamic_cast

Example: a failed cast

```
struct A { virtual ~A () {} };  
struct B : A { };  
struct C : B { int m; };
```

```
int test ()  
{  
    A obj;  
  
    A *a = &obj;  
  
    C *c = dynamic_cast<C *> (a);  
  
    return c->m;  
}
```

Before: No output, `c` is conjured

After:

```
dyncast-fail.cc:10:13: warning: dereference of NULL 'c' [CWE-476] [  
    10 |     return c->m;  
        |           ^  
dyncast-fail.cc:9:10: note: (1) following 'true' branch...  
     9 |     C *c = dynamic_cast<C *> (a);  
        |           ^  
dyncast-fail.cc:9:10: note: (2) ...to here  
     9 |     C *c = dynamic_cast<C *> (a);  
        |           ^  
dyncast-fail.cc:10:13: note: (3) dereference of NULL 'c'  
    10 |     return c->m;  
        |           ^
```

The rules: [expr.dynamic.cast]/9

/9.1 downcast: start at the **source subobject**, find the unique **Dst** derived from it, with a public path down to **Src**:

```
C obj;
```

```
A *a = &obj;
```

```
dynamic_cast<C *> (a);
```

```
A <- src: a
```

```
|
```

```
B
```

```
|
```

```
C <- dst and MD
```

/9.2 crosscast: start at the **most-derived object**, **Src** a public base of it, **Dst** an unambiguous public base:

```
struct D : C, S {};
```

```
D obj;
```

```
A *a = &obj;
```

```
dynamic_cast<S *> (a);
```

```
A <----- src: a
```

```
|
```

```
B      S <- dst
```

```
|
```

```
C      |
```

```
\
```

```
/
```

```
D <---- MD
```

/9.3: otherwise return null (or throw `std::bad_cast`)

A /9.1 corner case - PR126510

```
struct S { int s; constexpr virtual ~S () {} };  
struct C : virtual S { int c; };  
struct MD : private C { int d; };
```

```
constexpr bool f ()  
{  
    MD obj;  
    S *s = (S *) (C *) &obj;  
    C *c = dynamic_cast<C *> (s);  
    return c != nullptr && c == (C *) &obj;  
}
```

```
static_assert (f ()); // fails, should succeed
```

```
src: s -> S  
      | virtual  
dst ----> C  
      | private  
      MD
```

Runtime `__dynamic_cast`: succeeds

Analyzer (same hierarchy, at runtime):

```
pr126510.cc:12:19: warning: TRUE  
pr126510.cc:13:19: warning: TRUE
```

C++26 constexpr evaluator:

```
pr126510-constexpr.cc:13:18: error: static assertion failed
```

/9.1 only needs the `C` $\Rightarrow$ `S` path to be public, and exactly one `C` derives from this `S`.

The private `C` $\Rightarrow$ `MD` edge only matters for /9.2.

Implementation

The failed cast from earlier, in GIMPLE:

```
if (a.0 != 0B) ... // FE null check: events (1)-(2)
c = __dynamic_cast (a.0, &_ZTI1A, &_ZTI1C, 0);
      |           |           |           |
      |           |           |           +--> src2dst hint: ignored in the analyzer
      |           |           +-----> dst type: C (RTTI info object)
      |           +-----> src type: A (RTTI info object)
      +-----> vptr binding: &_ZTV1A + 16 -> most-derived type A
```

- a new `known_function` evaluates `[expr.dynamic.cast]/9` over the most-derived type's `BINFO`s
- found: a pointer into the object, otherwise null
- anything unexpected (unknown dynamic type, odd direct call): LHS stays conjured

Computing the result of dynamic_cast

```
#include "analyzer-decls.h"

struct A { virtual ~A () {} };
struct B : A { };
struct C : B { int m; };

void test ()
{
    C obj;
    obj.m = 50;
    A *a = &obj;
    C *c = dynamic_cast<C *> (a);
    __analyzer_eval (c->m == 50);
    __analyzer_eval (c == &obj);
}
```

Before:

```
dyncast-precision.cc:13:19: warning: UNKNOWN
dyncast-precision.cc:14:19: warning: UNKNOWN
```

After:

```
dyncast-precision.cc:13:19: warning: TRUE
dyncast-precision.cc:14:19: warning: TRUE
```

The result is an `offset_region` into `obj`, so it aliases the frontend's own field accesses.

We can prove `c->m == 50` as well, since we compute the actual object.

What's left

Out of scope for the project

- thunk `fixed_offset` before binding `this`
- small patch for recognizing `std::bad_cast`
- exception matching under mixed-language LTO

Interesting future work for C++

- `std::vector` and other containers
- iterators and invalidation
- `std::variant`, `std::unique_ptr`

Much of the standard library uses virtual calls, exceptions or `dynamic_cast` in some form, so this work was a prerequisite in order to model more complex C++ constructs.

Questions?

Thanks to David Malcolm and the gcc-patches reviewers.

Write-up: riogu.github.io/posts/gsoc-2026

Wiki: gcc.gnu.org/wiki/StaticAnalyzer/C++